

MIDISTRUCT VARIATIONS

Technical Documentation

Version 2.1.0

Acrosonus Mastering · Lua Script for REAPER DAW

Copyright © 2026 Acrosonus Mastering. All rights reserved.

1 · Overview

MIDISTRUCT Variations is a REAPER Lua script that analyzes the active MIDI take of a selected media item and generates eight distinct musical variations, each placed in its own REAPER Take. The script is designed for professional music production workflows — from subtle humanization (Madness 1) to radical creative mutation (Madness 10).

Every variation is produced non-destructively: the original Take 1 is never modified and is automatically restored as the active take after generation.

i Scope Only the first selected MediaItem is processed. The active take of that item must be a MIDI take. If no item is selected, or if the take is not MIDI, an error dialog is shown and the script exits without writing any data.

1.1 Output Summary

Take	Name	Algorithm Family
1	Original (unchanged)	Source — read-only
2	Syncopation [Mx]	Rhythmic displacement
3	Smart Velocity [Mx]	Dynamic shaping
4	Octave & Simplify [Mx]	Reduction + pitch color
5	Strummer [Mx]	Chord sweep / arpeggio
6	Gate / Staccato [Mx]	Duration truncation
7	Power Harmonizer [Mx]	Harmonic layering + CC riser
8	Ghost Delay [Mx]	MIDI echo / tape delay
9	Melodic Mirror [Mx]	Pitch axis inversion

[Mx] = Madness level stamped in the take name for visual reference (e.g. [M7]).

2 · Installation & Usage

2.1 Installation

- Copy MIDISTRUCT_Variations_v2.lua into your REAPER Scripts folder.
- In REAPER: Actions → Show action list → Load... → select the file.
- Optionally assign a keyboard shortcut in the Action List for one-key access.

2.2 Workflow

4. Select a MIDI item in the Arrange view.
5. Run the script (keyboard shortcut or double-click in Action List).
6. The Madness Dial dialog appears, pre-filled with your last used value.
7. Enter a value from 1 to 10 and press Enter (or click OK).
8. Eight new Takes are created. Right-click the item → Take to audition each.

 **Pro tip** Assign a keyboard shortcut (e.g. Alt+V). Each press regenerates with a new random seed — the Madness level is remembered, so you can rapid-fire variations without touching the dialog.

2.3 Requirements

- REAPER 6.0 or later (any OS)
- No external Lua libraries — uses only the standard REAPER API
- No VST or audio plugins required

3 · Architecture

3.1 Execution Flow

The script follows a strict sequential pipeline on each run:

```

math.randomseed()           ← high-entropy seed (os.time + reaper.time_precise)
load_last_madness()         ← read ExtState from reaper.ini
ask_madness_level()         ← GetUserInputs dialog (pre-filled)
save_last_madness()         ← persist new value to reaper.ini
apply_madness(level)        ← scale all CFG parameters
GetSelectedMediaItem()      ← validate item
GetActiveTake() + TakeIsMIDI() ← validate take
read_notes()                ← MIDI_GetNote loop → Lua table
get_ppq_per_beat()          ← derive PPQ from tempo map
Undo_BeginBlock()
for each variation def:
    AddTakeToMediaItem()     ← create blank MIDI take
    copy_notes(src)          ← deep copy (isolation)
    def.fn(notes, ppqbeat)   ← run algorithm
    write_notes()            ← MIDI_DisableSort + InsertNote loop + MIDI_Sort
    def.after() (optional)   ← e.g. write CC riser
SetMediaItemInfo_Value(I_CURTAKE, 0) ← restore original as active
Undo_EndBlock()
UpdateItemInProject() + UpdateArrange()

```

3.2 Configuration Table (CFG)

All algorithm parameters are declared in a single top-level CFG table and overridden at runtime by `apply_madness()`. Modifying CFG defaults changes the behavior at level 5 (the midpoint).

CFG Key	Default	Description
SYNC_CHANCE	0.30	Fraction of notes displaced in Syncopation
SYNC_HUMANIZE_MAX	10	Max $\pm$ tick drift on displaced notes
SYNC_MICRO_MAX	3	Max $\pm$ tick micro-drift on untouched notes
VEL_DOWNBEAT_BOOST	1.15	Velocity multiplier on downbeat notes
VEL_OFFBEAT_CUT	0.80	Velocity multiplier on off-beat notes
VEL_ACCENT_SWING	0.08	Depth of sinusoidal swell per beat
VEL_HUMANIZE_MAX	4	Max $\pm$ velocity units of final humanization
SIMP_REMOVE_CHANCE	0.20	Base probability to remove a note
SIMP_SHORT_BIAS	1.6	Removal weight for short notes
SIMP_OCTAVE_CHANCE	0.15	Fraction of notes shifted ± 1 octave
SIMP_LEGATO_GLUE	0.92	Note stretch to X% of gap to next note
STRUM_TICK_OFFSET	15	Ticks between strum steps
STRUM_VEL_DECAY	0.92	Velocity multiplier per strum step
STRUM_HUMAN_MAX	2	$\pm$ tick micro-jitter per strum step
GATE_DURATION_RATIO	0.25	Note length cut ratio (1.0 = unchanged)
GATE_VEL_BOOST	1.20	Velocity boost to compensate short attack
GATE_MIN_TICKS	20	Minimum note length (prevents silent notes)
HARM_FIFTH_SEMIS	7	Semitones for upper harmony (7 = 5th)
HARM_SUB_SEMIS	-12	Semitones for sub-bass copy (-12 = octave)
HARM_FIFTH_VEL_MULT	0.80	Fifth layer velocity ratio vs. original
HARM_SUB_VEL_MULT	0.70	Sub-bass layer velocity ratio vs. original
CC_RISER_ENABLE	true	Write CC filter riser into Harmonizer take
CC_RISER_NUM	74	CC number (74=filter cutoff, 1=modwheel)
CC_RISER_STEPS	32	CC riser resolution (events)
GHOST_REPEATS	3	Number of echo copies per note
GHOST_INTERVAL_DIV	2	Echo interval = ppqbeat / N (2=eighth)
GHOST_VEL_DECAY	0.50	Velocity multiplier per echo
GHOST_PITCH_DRIFT	0	Semitones added per echo (tape drift)
MIRROR_AXIS_MODE	median	Inversion axis: median pitch or C4 (60)
MIRROR_FOLD_OCTAVE	true	Fold vs. clamp out-of-range mirror notes

3.3 Madness Dial — Parameter Scaling

The `apply_madness(level)` function normalizes the 1–10 input to a 0.0–1.0 `t` value and linearly interpolates each parameter between a "calm" extreme and a "chaos" extreme:

```
local t = (level - 1) / 9.0
local function dial(calm, chaos) return calm + (chaos - calm) * t end

CFG.SYNC_CHANCE          = dial(0.10, 0.80)    -- 10% → 80%
CFG.GATE_DURATION_RATIO = dial(0.40, 0.08)    -- 40% → 8% cut
CFG.GHOST_REPEATS       = floor(dial(2, 5))    -- 2 → 5 echoes
CFG.GHOST_PITCH_DRIFT   = (t >= 0.6) and 1 or 0 -- drift above level 6
```

3.4 ExtState Persistence

The last-used Madness level is stored in REAPER's internal key-value store using:

```
-- Write (persist = true → survives REAPER restart)
reaper.SetExtState("AcrosonusMastering", "MIDISTRUCT_madness", "8", true)

-- Read (returns "" if key does not exist)
reaper.GetExtState("AcrosonusMastering", "MIDISTRUCT_madness")
```

Data is written to `reaper.ini` under the `[ext]` section. The `persist` flag ensures the value survives REAPER restarts. If the key is missing or non-numeric, the script defaults to level 5.

4 · Variation Algorithms

4.1 Syncopation

Rhythmic displacement that shifts a configurable fraction of notes forward by exactly half a beat (eighth note), with an additional layer of micro-humanization applied to all notes.

```
half_beat = ppqbeat / 2

for each note:
  if random() < SYNC_CHANCE:
    new_start = note.start + half_beat + rand(±SYNC_HUMANIZE_MAX)
  else:
    new_start = note.start + rand(±SYNC_MICRO_MAX)    -- micro drift
  note.end = new_start + original_duration

re-sort by startppq
```

Madness scaling: SYNC_CHANCE 10%→80%, SYNC_HUMANIZE_MAX 3→30 ticks, SYNC_MICRO_MAX 1→15 ticks.

4.2 Smart Velocity

Beat-aware dynamic shaping. Notes are classified by their position within a beat ($\text{phase} = \text{startppq} \bmod \text{ppqbeat} / \text{ppqbeat}$), then two transformations are applied on top of each other:

- Beat-phase gate: downbeats ($\text{phase} < 0.08$ or $\text{phase} > 0.92$) receive a boost; off-beats receive a cut.
- Sinusoidal swell: a cosine curve adds a subtle $\pm 8\%$ swell across the entire beat, peaking at the downbeat boundary and dipping at the "and".
- Final humanization: ± 4 velocity units of random noise prevents machine-gun uniformity.

```
phase = (note.start % ppqbeat) / ppqbeat  -- 0.0 → 1.0

if near_downbeat(phase):
    vel *= VEL_DOWNBEAT_BOOST
else:
    vel *= VEL_OFFBEAT_CUT

swell = 1 + VEL_ACCENT_SWING * cos(phase * 2π)
vel   = vel * swell + rand(±VEL_HUMANIZE_MAX)
vel   = clamp(vel, 1, 127)
```

4.3 Octave & Simplify

Three sequential passes reduce and color the note set:

Pass 1 — Weighted Removal

Each note's removal probability scales inversely with its length relative to the median note length. Short notes are $\text{SIMP_SHORT_BIAS} \times$ more likely to be removed than median-length notes. Probability is capped at 85% to guarantee at least some short notes survive.

```
len_ratio   = note_length / median_length
remove_prob = SIMP_REMOVE_CHANCE * (SIMP_SHORT_BIAS / max(len_ratio, 0.1))
remove_prob = min(remove_prob, 0.85)
```

Pass 2 — Directional Octave Shift

$\text{SIMP_OCTAVE_CHANCE}$ fraction of surviving notes are shifted by ± 1 octave. Notes below MIDI 60 (C4) prefer to shift upward; notes above prefer to shift downward. A 10% flip probability allows occasional contrary-direction shifts for variety.

Pass 3 — Legato-Glue

Each surviving note is extended to $\text{SIMP_LEGATO_GLUE} \times (\text{gap to next note})$, creating a more connected, sustained feel. Notes are never shortened by this operation.

4.4 Strummer

Groups notes that share the same start position (within a 2-tick tolerance to handle near-simultaneous notes) and staggers them by STRUM_TICK_OFFSET ticks, simulating a guitar strum or EDM chord sweep.

- Notes within each group are sorted by pitch (up, down, or alternating per group).
- Velocity decays across the strum: $vel \times STRUM_VEL_DECAY^{(step-1)}$.
- Each step receives $\pm STRUM_HUMAN_MAX$ ticks of random micro-jitter for organic feel.

```
for each chord group:
    sort notes by pitch (direction alternates per group)
    for step = 1..N:
        jitter      = rand( $\pm STRUM\_HUMAN\_MAX$ )
        offset      = (step-1)  $\times$  STRUM_TICK_OFFSET + jitter
        vel_scale   = STRUM_VEL_DECAY ^ (step-1)
        new_start   = group.anchor + max(offset, 0)
```

4.5 Gate / Staccato

Truncates every note to GATE_DURATION_RATIO of its original length, with a minimum floor of GATE_MIN_TICKS to prevent inaudible micro-notes. Velocity is boosted to compensate for the reduced sustain, modeling the physics of a plucked or struck attack.

```
new_dur = max(original_dur  $\times$  GATE_DURATION_RATIO, GATE_MIN_TICKS)
note.end = note.start + new_dur
note.vel = clamp(note.vel  $\times$  GATE_VEL_BOOST, 1, 127)
```

At Madness 10: duration cut to 8% (≈ 77 ticks at 960 PPQ), velocity boosted by $\times 1.45$ — transforms any pad or string into a hyper-aggressive pluck.

4.6 Power Harmonizer

Doubles every original note at two additional intervals to create an instant "power unison" stack, as used in Big Room EDM, Avicii-style leads, and cinematic synths.

Layer	Interval	Velocity
Original	± 0 semitones	100% (unchanged)
Perfect Fifth	+7 semitones	HARM_FIFTH_VEL_MULT (default 80%)
Sub-Bass	-12 semitones	HARM_SUB_VEL_MULT (default 70%)

Out-of-range notes are silently skipped (fifth > 127 , sub < 0). Both layers can be independently enabled via HARM_ADD_FIFTH and HARM_ADD_SUB.

CC Riser

After the notes are written, a CC smoothstep riser is stamped into the same take. The riser uses a cubic ease-in-out curve ($t^2(3-2t)$) rather than a linear ramp, so the filter sweep accelerates and decelerates naturally:

```

for i = 0..CC_RISER_STEPS:
    t          = i / CC_RISER_STEPS
    smooth_t   = t² × (3 - 2t)          -- smoothstep
    ppq_pos    = total_item_ppq × t
    cc_val     = CC_RISER_START + range × smooth_t
    MIDI_InsertCC(take, ppq_pos, 0xB0, CC_RISER_NUM, cc_val)

```

4.7 Ghost Delay

A pure MIDI echo engine that creates GHOST_REPEATS copies of each original note, each shifted forward by one interval (ppqbeat / GHOST_INTERVAL_DIV), with decaying velocity and optional pitch drift.

- Echo interval: at GHOST_INTERVAL_DIV = 2 → eighth note; 4 → sixteenth; 3 → eighth-note triplet.
- Velocity decay: $vel \times GHOST_VEL_DECAY^{repeat}$ — at 0.50 decay, 3 echoes reach 50% → 25% → 12.5%.
- Duration decay: each echo is shortened by $\times 0.9^{rep}$ to simulate natural tail shortening.
- Pitch drift: GHOST_PITCH_DRIFT semitones are accumulated across repeats (activated at Madness ≥ 6).

```

for each original note:
    keep original (if GHOST_KEEP_ORIGINAL)
    vel = note.vel ; acc_drift = 0
    for rep = 1..GHOST_REPEATS:
        vel          = vel × GHOST_VEL_DECAY
        acc_drift += GHOST_PITCH_DRIFT
        echo_start = note.start + interval × rep
        echo_dur   = original_dur × (0.9 ^ rep)
        echo_pitch = clamp(note.pitch + acc_drift, 0, 127)

```

Pitch drift creates a "dub tape delay" effect: at +1 semi/repeat over 3 echoes, the echoes rise by a minor third total — great for Trap, Dubstep, and cinematic sound design.

4.8 Melodic Mirror

Applies a classical pitch-axis inversion (used by Bach, Hindemith, and modern EDM producers) to generate an instant counter-melody that is harmonically complementary to the original.

Inversion Formula

```

mirrored_pitch = 2 × axis - original_pitch

-- Example: axis = 64 (E4)
-- Original: C4 (60) → mirrored: 2×64 - 60 = 68 (Ab4) ✓ same interval, opposite
direction
-- Original: G4 (67) → mirrored: 2×64 - 67 = 61 (Db4) ✓

```

Axis Modes

Mode	Axis Value	Best for
median	Median pitch of source notes	Melodic lines (contextual, stays in range)
center	MIDI 60 (C4)	Fixed-key compositions, theoretical work

Out-of-Range Handling (Octave Fold)

When `MIRROR_FOLD_OCTAVE` is true, notes that fall outside 0–127 are shifted by octaves until they re-enter the valid range. This preserves harmonic interval identity — unlike a hard clamp, which would collapse multiple distinct pitches onto a single value.

```
-- Fold: preserves interval color
while pitch < 0 do pitch = pitch + 12 end
while pitch > 127 do pitch = pitch - 12 end

-- Clamp: destroys interval color (MIRROR_FOLD_OCTAVE = false)
pitch = max(0, min(127, pitch))
```

5 · MIDI I/O Reference

5.1 Reading Notes

All notes are read in a sequential index loop using `MIDI_GetNote`. The loop exits when `retval` is false (past the last note index). Notes are stored as Lua tables in a flat array.

```
local notes, idx = {}, 0
while true do
  local ok, sel, mut, s, e, ch, p, v = reaper.MIDI_GetNote(take, idx)
  if not ok then break end
  notes[#notes+1] = { selected=sel, muted=mut,
                    startppq=s, endppq=e,
                    chan=ch, pitch=p, vel=v }

  idx = idx + 1
end
```

5.2 Writing Notes

`MIDI_DisableSort` is called before the first `MIDI_InsertNote`, and `MIDI_Sort` is called once after the last insertion. This is required by the REAPER API — inserting notes without disabling sort first causes $O(n^2)$ performance degradation on large takes.

```
reaper.MIDI_DisableSort(take)
for _, n in ipairs(notes) do
  reaper.MIDI_InsertNote(take, n.selected, n.muted,
    n.startppq, n.endppq, n.chan, n.pitch, n.vel, false)
end
```

```
reaper.MIDI_Sort(take)      -- sort once at the end
```

5.3 PPQ-per-Beat Derivation

Rather than assuming a fixed 960 PPQ, the script derives the true PPQ-per-beat from the project's tempo map using a round-trip through REAPER's time/beat conversion API. This ensures correctness at any tempo and with tempo automation:

```
pos0      = MIDI_GetProjTimeFromPPQPos(take, 0)
beats     = TimeMap2_timeToBeats(0, pos0)
tlbeat    = TimeMap2_beatsToTime(0, beats + 1)
ppqbeat   = MIDI_GetPPQPosFromProjTime(take, tlbeat)
-- Fallback to 960 if result is degenerate
if ppqbeat <= 0 then ppqbeat = 960 end
```

5.4 Note Isolation (Deep Copy)

Each variation receives an independent deep copy of the source notes table. This prevents any algorithm from mutating the shared source and contaminating subsequent takes:

```
local function copy_notes(src)
  local dst = {}
  for i, n in ipairs(src) do
    dst[i] = { selected=n.selected, muted=n.muted,
               startppq=n.startppq, endppq=n.endppq,
               chan=n.chan, pitch=n.pitch, vel=n.vel }
  end
  return dst
end
```

6 · Randomization

The RNG is seeded once at startup with a two-component value that combines a second-precision timestamp with a sub-millisecond counter from REAPER's high-resolution clock. This ensures a unique seed on every run, even when the script is triggered multiple times within the same second.

```
math.randomseed(os.time() + math.floor(reaper.time_precise() * 100000))
for _ = 1, 10 do math.random() end  -- warm up LCG state
```

The 10-iteration warm-up discards the first outputs of Lua's linear congruential generator, which are known to be poorly distributed on low-entropy seeds.

i Determinism Two runs with the same Madness level will produce different variations because the RNG seed is time-dependent. There is no way to reproduce an exact result — intentional design for creative exploration.

7 · Undo / Redo

The entire script execution is wrapped in a single undo block. Ctrl+Z will remove all eight generated takes in one operation, atomically restoring the item to its pre-run state:

```
reaper.Undo_BeginBlock()
-- ... all take creation and note writing ...
reaper.Undo_EndBlock("MIDISTRUCT Variations v2.1 - 8 takes, Madness N", -1)
```

The `-1` flag as the last argument to `Undo_EndBlock` instructs REAPER to automatically determine the undo scope from the operations performed within the block.

8 · Known Limitations

- Only the first selected item is processed. Multi-item batch processing is not supported in v2.1.
- MIDI CC, pitch-bend, and aftertouch events from the source take are not copied to variations (notes only).
- The Ghost Delay algorithm does not check for PPQ overflow — very long repeats near the end of a short item may place echoes beyond the item boundary. REAPER will clip them visually but the data is valid.
- Melodic Mirror timing is untouched. If the original melody has strong rhythmic identity, the mirror will share the same rhythm — which is often desirable but worth noting.
- `GHOST_INTERVAL_DIV` values that produce a non-integer `ppqbeat/N` are silently floored by `math.floor()`.

9 · Version History

Version	Date	Changes
1.0.0	2026-01	Initial release — 3 variations (Syncopation, Smart Velocity, Octave & Simplify)
2.0.0	2026-01	Added Strummer (4), Gate/Staccato (5), Power Harmonizer (6) + CC riser. Madness Dial. Deep-copy isolation. PPQ derivation from tempo map.
2.1.0	2026-01	Added Ghost Delay (7), Melodic Mirror (8). ExtState persistence. Strummer micro-jitter. Pitch drift in Ghost Delay. Octave-fold in Mirror.

10 · Glossary

Term	Definition
PPQ	Pulses Per Quarter note — the internal timing resolution of a MIDI take
Downbeat	A note whose start position is within <code>VEL_DOWNBEAT_NEAR</code> fraction of a beat boundary

Term	Definition
Smoothstep	Cubic interpolation curve $t^2(3-2t)$ that eases in and out (no abrupt start/end)
Deep copy	A full duplication of a data structure with no shared references to the original
Legato-glue	Extending a note's end to approach the start of the next note, reducing silence gaps
ExtState	REAPER's key-value persistence store, written to reaper.ini
Octave fold	Shifting an out-of-range pitch by ± 12 semitones until it falls within 0–127
Pitch drift	Accumulated semitone offset applied to each successive echo in Ghost Delay
LCG	Linear Congruential Generator — Lua's built-in pseudo-random number algorithm